Automated Design and Stability Analysis of a Statically and Dynamically Stable Jet Trainer

Steve Mathew Kiran

Department of Mechanical Engineering, University of Windsor

MECH-4671: Aerodynamics and Performance

github.com/SteveKiran7071/AVL-Neural-Network-development

August 5, 2026

Table of contents

List of Figures 3

List of Tables 4

Abstract 5

Introduction 5

Literature Review and Background 6

Problem Definition 6

Theory 7

Cruise Performance and the Drag Polar 7

Section and Wing Lift-Curve Slopes 8

Static Stability and the Aerodynamic Center 9

Dynamic-Mode Approximations 9

Optimization Formulation 10

Methodology 11

Automated Design Pipeline 11

Hand-Run Section and Aerodynamic-Center Analyses 12

Verification 12

Results and Discussions 13

Two-Dimensional Section Analysis of the NACA 2312 13

Wing Aerodynamic Center and Wing-Alone Stability 16

Final Design, Trim, and Static Stability 18

Dynamic Stability 22

Optimization Convergence and Surrogate Assessment 24

Conclusion 28

References 29

Appendix A – AVL and XFoil Submission Files 30

Appendix B – Developed Code 31

List of Figures

Figure 1: XFoil 7.00 Polar of the NACA 2312 at Re = 4.6×10^6 and M = 0.5 12

Figure 2: Section Pressure Distribution at the Cruise Trim Angle ($\alpha = 6.76°$) 13

Figure 3: Wing-Only AVL Model for the Aerodynamic-Center Exercise 14

Figure 4: AVL Geometry of the Final Design (Hardcopy Output) 14

Figure 5: Trefftz-Plane Loading at Cruise Trim 17

Figure 6: Evolutionary-Algorithm Best-Fitness Convergence Across Archived Runs 20

Figure 7: Surrogate-Predicted Versus Real-AVL Fitness (Parity Plot) 21

Figure 8: Real-AVL Re-Validation of Surrogate-Selected Elites During the Surrogate-Assisted Search 22

List of Tables

Table 1: XFoil Polar Summary for the NACA 2312 (Re = 4.6×10^6, M = 0.5, Ncrit = 9) 11

Table 2: Complete Optimized Design Vector (Eighteen Variables) 15

Table 3: Trimmed Cruise State and Performance of the Final Design 16

Table 4: Dynamic Modes of the Final Design (AVL Eigenvalue Analysis, Labels Eigenvector-Confirmed) 18

Table 5: Classical Approximations Versus AVL Solutions 19

Abstract

The conceptual design of a statically and dynamically stable jet trainer was completed for a 4000 kg aircraft cruising at Mach 0.5 at 11.0 km. Rather than iterating by hand, an automated pipeline evaluated every candidate in a vortex-lattice solver, an evolutionary algorithm searched the eighteen-variable design space, and a neural-network surrogate was assessed as a potential accelerator. The optimized aircraft trimmed at a lift-to-drag ratio of 14.1 with a static margin of 0.362 and a stall margin of 3.24 degrees, and every dynamic mode was stable except the spiral, whose 729-second divergence satisfied the requirement's exception. The design matched the closed-form optimum of the prescribed drag polar, confirming the search located the true performance ceiling rather than a merely feasible design.

Introduction

Jet trainers must be docile enough for student pilots yet retain the speed and handling of the aircraft they emulate — a balance fixed by a few early geometric choices, historically iterated by hand. This project completed that conceptual design for a 4000 kg trainer (cruise Mach 0.5 at 11.0 km) without hand iteration: AVL evaluated every candidate as ground truth, an evolutionary algorithm (EA) searched the eighteen-variable design space, and a neural-network surrogate was assessed as an accelerator. The prescribed NACA 2312 section was characterized in XFoil and the wing's aerodynamic center extracted from a wing-only model, as required.

Across four seeds the pipeline converged to one design: L/D = 14.1 (the closed-form optimum of the prescribed drag polar), static margin 0.362, stall margin 3.24°, and stable dynamics in every mode except an allowed slow spiral. Every number here was regenerated by a verified AVL run after a mass-file flight-density defect was found and fixed.

Literature Review and Background

The analytical backbone is classical: the finite-wing and Prandtl-Glauert corrections and the drag-polar description of cruise efficiency follow Anderson (2017); the aerodynamic center, neutral point and static margin, and the closed-form dynamic-mode approximations follow Etkin and Reid (1996) and Nelson (1998); tail volume coefficients and trainer-class geometry ranges are from Raymer (2018).

The tools are established: XFoil couples a panel method with an integral boundary-layer formulation and a Karman-Tsien compressibility correction (Drela, 1989), and AVL implements an extended vortex-lattice method returning force coefficients, stability derivatives, and eigenvalues (Drela & Youngren, n.d.). Evolutionary algorithms are an accepted choice for design-space search over non-smooth, constraint-bounded objectives (Raymer, 2018).

Problem Definition

The task was to design a 4000 kg jet trainer cruising at Mach 0.5 at 11.0 km (density 0.364 kg/m³, airspeed 148 m/s, dynamic pressure 3960 Pa, weight 39,200 N, mean-chord Reynolds number $\approx 4.6\times10^{6}$). The wing section was the prescribed NACA 2312, and the design had to be statically and dynamically stable. Deliverables: wing design and placement, tail sizing, trim at cruise, and quantitative evidence of static and dynamic stability.

Several models were fixed by the assignment. Cruise performance was judged against the prescribed parabolic drag polar (zero-lift drag 0.03, Oswald efficiency a function of aspect ratio alone, developed in the Theory). The mass was 60% fuselage — twenty 120 kg point masses — and 20% per wing at $y = \pm b/6$; the inertias used throughout (I_{xx} = 5890, I_{yy} = 11,200, I_{zz} = 16,400 kg·m²) follow from this layout.

Three stability requirements applied: the aircraft had to be statically stable (center of gravity ahead of the neutral point); every dynamic mode had to be stable, except that a divergent spiral was permitted if its time constant was at least 20 s; and no wing section could exceed the 10° angle-of-attack limit anywhere along the span — a check applied to the trim angle of attack plus the wing incidence, not the body angle alone.

Within these constraints the optimizer set eighteen design variables: wing aspect ratio, taper, quarter-chord sweep, dihedral, incidence, area, and root leading-edge location; aileron span start and chord fraction; each tail's volume coefficient, aspect ratio, sweep, and arm; and the fuselage row spacing. The complete optimized vector, with bound status, is in Appendix A.

Theory

The relations below are stated in the exact form the pipeline implements (Anderson, 2017; Etkin & Reid, 1996; Nelson, 1998); each is compared against the AVL solutions in the Results and Discussions.

Cruise Performance and the Drag Polar

The assignment prescribed that cruise performance be evaluated against the parabolic drag polar

$$C_D = C_{D,0} + \frac{C_L^2}{\pi e_0 AR} \tag{1}$$

where CD,0 = 0.03 is the prescribed zero-lift drag coefficient, AR is the wing aspect ratio, and e0 is the Oswald efficiency factor, itself prescribed as a function of the aspect ratio alone:

$$e_0 = 1.78(1 - 0.045AR^{0.68}) - 0.64 \tag{2}$$

Because the polar depends only on the lift coefficient and the aspect ratio, the best attainable cruise efficiency has a closed form. Maximizing the lift-to-drag ratio over the lift coefficient gives

$$(\frac{L}{D})_{max} = \frac{1}{2}\sqrt{\frac{\pi e_0 AR}{C_{D,0}}} \tag{3}$$

attained at the optimum lift coefficient

$$C_L^* = \sqrt{C_{D,0}\pi e_0 AR} \tag{4}$$

Equations (3) and (4) are the optimizer's yardstick: the ceiling grows only with aspect ratio, independent of every other variable, and the wing area must be sized to fly the optimum lift coefficient at cruise.

Section and Wing Lift-Curve Slopes

The three-dimensional results were checked against the classical section-to-aircraft chain. Thin-airfoil theory gives an incompressible section slope of 2π/rad; the Prandtl-Glauert transformation raises it to

$$a_0 = \frac{2\pi}{\sqrt{1 - M_\infty^2}} \tag{5}$$

where M∞ is the freestream Mach number. Helmbold's relation then corrects a finite surface of aspect ratio AR for the induced downwash of the trailing-vortex system (Anderson, 2017):

$$a_t = \frac{2\pi AR}{2 + \sqrt{AR^2\beta^2 + 4}} \tag{6}$$

where $\beta^2 = 1 - M_\infty^2$. Finally, the horizontal tail sees a reduced angle-of-attack response through the wing's downwash field, with the classical gradient

$$\frac{d\varepsilon}{d\alpha} = \frac{2a_w}{\pi AR} \tag{7}$$

where aw is the wing lift-curve slope; Eqs. (5)–(7) feed both the XFoil slope check and the analytical neutral-point estimate compared with AVL.

Static Stability and the Aerodynamic Center

The wing's aerodynamic center — where the wing pitching moment is independent of lift — was extracted from a wing-only model, per the assignment's procedure, as

$$x_{AC} = x_{ref} - \bar{c}\frac{dC_m}{dC_L} \tag{8}$$

where xref is the moment reference point (the aircraft center of gravity) and $\bar{c}$ is the mean aerodynamic chord. For the complete aircraft, the horizontal tail moves the neutral point aft of the wing aerodynamic center (Etkin & Reid, 1996; Nelson, 1998):

$$x_{np} = x_{AC,w} + V_H \frac{a_t}{a_w}(1 - \frac{d\varepsilon}{d\alpha})\bar{c} \tag{9}$$

where VH is the horizontal-tail volume coefficient and at and aw the tail and wing lift-curve slopes, with the downwash gradient of Eq. (7). Static stability is measured by the static margin,

$$SM = \frac{x_{np} - x_{cg}}{\bar{c}} \tag{10}$$

which must be positive for the center of gravity to lie ahead of the neutral point.

Dynamic-Mode Approximations

AVL's eigenvalue analysis is authoritative; the classical approximations below (Etkin & Reid, 1996; Nelson, 1998) are an independent check. The short-period natural frequency follows from the pitch stiffness,

$$\omega_{n,sp} \approx \sqrt{\frac{-C_{m\alpha}\bar{q}S\bar{c}}{I_{yy}}} \tag{11}$$

and its damping ratio from

$$2\zeta\omega_n \approx -M_q + \frac{Z_\alpha}{mV} \tag{12}$$

where Mq = q̄Sc̄(c̄/2V)Cmq/Iyy is the dimensional pitch-damping derivative and Zα the vertical-force derivative. The phugoid follows Lanchester's approximation,

$$\omega_{n,ph} \approx \frac{\sqrt{2}g}{V}, \quad \zeta_{ph} \approx \frac{1}{\sqrt{2}}\frac{C_D}{C_L} \tag{13}$$

The roll mode is a single-degree-of-freedom lag with root

$$\lambda_{roll} \approx \frac{\bar{q}Sb\,\frac{b}{2V}\,C_{lp}}{I_{xx}} \tag{14}$$

and the Dutch-roll natural frequency follows from the weathercock stiffness,

$$\omega_{n,dr} \approx \sqrt{\frac{\bar{q}SbC_{n\beta}}{I_{zz}}} \tag{15}$$

The spiral mode has no useful frequency approximation; the classical criterion is that it converges when Clβ·Cnr exceeds Clr·Cnβ.

Optimization Formulation

The search maximizes trimmed cruise L/D minus proportional penalties (weight 100) for each violated requirement — negative static margin, an unstable mode outside the spiral exception, spiral time constant below 20 s, negative stall margin, or failure to trim — plus, for feasible designs, a saturating stability-margin bonus capped at +0.05 that acts purely as a tie-break. A multilayer-perceptron surrogate of this fitness was investigated to reduce AVL calls.

Methodology

Automated Design Pipeline

All analyses ran through one Python pipeline driving AVL 3.52 as a subprocess: each design vector is expanded into AVL geometry, mass, and run-case files; AVL trims the aircraft at cruise; coefficients, derivatives, and eigenvalues are parsed; the Problem Definition checks are applied; and the outcome is condensed into the scalar fitness above. The AVL artifacts are catalogued in Appendix A, and the three core modules — the design-evaluation funnel, the fitness function, and the evolutionary algorithm — are reproduced in Appendix C.

The evolutionary search evolved a population of 40 for 60 generations with four parallel workers, every evaluation a real AVL run, repeated from four seeds. The surrogate study trained a PyTorch multilayer perceptron on a 2000-sample Latin-hypercube dataset of real AVL labels and embedded it in the same loop with one safeguard: every proposed improvement was re-evaluated by real AVL before acceptance. The overnight sequence — dataset, training, surrogate search, pure-AVL cross-check — ran unattended in about two hours.

Because the search is automated, the design steps a manual process would iterate are handled as optimizer variables rather than separate procedures: wing aspect ratio, taper, quarter-chord sweep, dihedral, incidence, area, and root placement; aileron span start and chord fraction (control-surface sizing); and each tail's volume coefficient, aspect ratio, sweep, and arm (tail sizing). AVL evaluates the resulting geometry at cruise, and the trim, static-margin, stall, and dynamic-mode checks of the Problem Definition decide feasibility. The wing-alone aerodynamic-center study and the analytical-versus-numerical stability comparison a manual method performs by hand appear in the Results.

Hand-Run Section and Aerodynamic-Center Analyses

Two analyses were run by hand, with their exact command sequences preserved in the submission's instruction file (Appendix A): the wing-only AVL model for the aerodynamic center — tails removed, reference quantities unchanged — evaluated by both a finite-difference sweep and the stability-derivative ratio of Eq. (8); and the XFoil 7.00 section study of the NACA 2312 at Re = 4.6×10^{6}, Mach 0.5, Ncrit = 9, producing the polar and the cruise-point pressure distribution.

Verification

Verification was treated as part of the methodology, in five layers: (a) 46 pytest unit tests covering file generation, parsing, and the stability and optimization logic without AVL; (b) a live AVL round trip confirming trim convergence, derivative parsing, and eigenvalue extraction; (c) a cross-check of the optimum against the closed form of Eqs. (3) and (4); (d) the aerodynamic center by two independent methods against AVL's printed neutral point; and (e) the classical approximations of Eqs. (11)–(15) against the AVL eigenvalues.

Two defects this process caught are worth recording. AVL applies the mass-file density as the flight density in the dynamic-mode solution; an early version supplied sea-level density, and all published eigenvalues postdate the fix to the cruise value (0.36392 kg/m^{3}). And the frequency-ordering heuristic mislabeled the short-period and Dutch-roll modes — their frequencies are nearly identical — so the labels reported here were confirmed from the eigenvectors. Neither defect changed a pass/fail outcome; both would have silently skewed the numbers.

Results and Discussions

Results follow the order the analysis was built: section study, wing-only aerodynamic-center exercise, full-aircraft trim and static stability, dynamic modes, and the optimizer and its surrogate. Every value is a re-verified pipeline output.

Two-Dimensional Section Analysis of the NACA 2312

Table 1 summarizes the XFoil polar of the NACA 2312 at the cruise Reynolds number of 4.6×10^6 and Mach 0.5; the full polar is Figure 1 and the cruise-point pressure distribution Figure 2. The measured lift-curve slope of 0.135/deg (7.72/rad) sits close to the compressibility-corrected thin-airfoil value $2\pi/\beta = 7.26$/rad from Eq. (5); the zero-lift angle (−1.97°) matches the 2% camber, and the quarter-chord moment (≈ −0.050) the thin-airfoil camber moment.

Table 1

XFoil Polar Summary for the NACA 2312 (Re = 4.6×10⁶, M = 0.5, Ncrit = 9)

Quantity	Value	Reference check
Lift-curve slope (linear range)	0.135/deg = 7.72/rad	$2\pi/\beta = 7.26$/rad (thin airfoil + compressibility)
Zero-lift angle	−1.97°	≈ −2° for 2% camber
cm at c/4 (linear range)	≈ −0.050	thin-airfoil camber moment
cl,max and stall angle	1.51 at 10.5°	assignment's 10° limit ≈ 2D stall
Cruise section point ($\alpha = 6.76°$)	cl = 1.16, cd = 0.0097, cm = −0.037	3.7° below stall onset

Profile cd near cruise	0.006–0.010	aircraft CD,0 = 0.03 (≈5× larger)

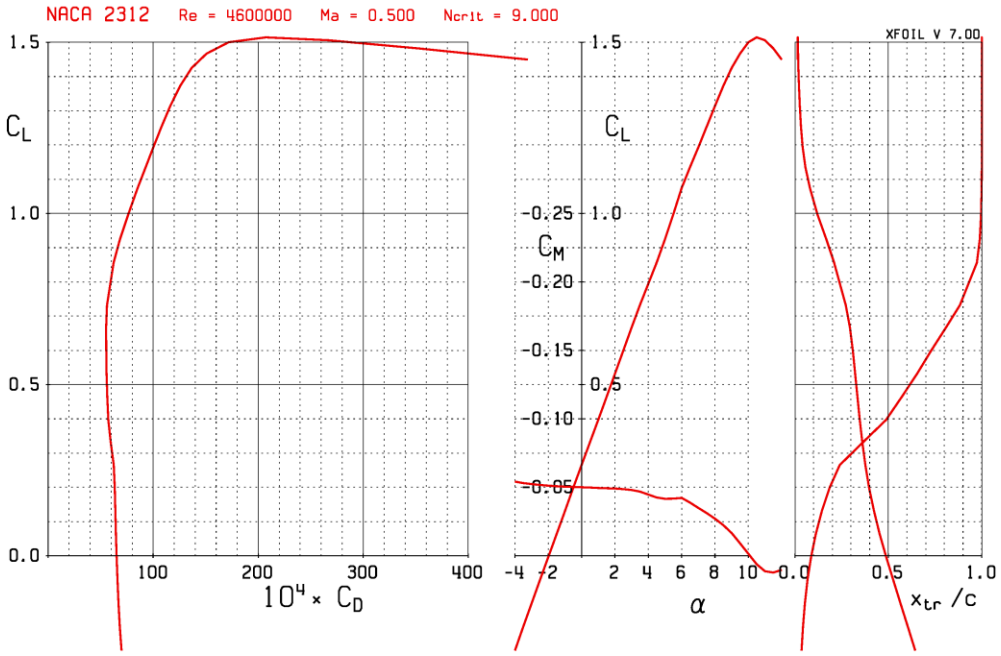
Note. Post-stall rows of the polar are qualitative given the approximate Karman-Tsien correction at M = 0.5.

Two findings matter at aircraft level. The section stalls at 10.5° with cl,max = 1.51, so the assignment's 10° limit nearly coincides with two-dimensional stall and satisfying it keeps every station unstalled. At the cruise section angle of 6.76° the section carries cl = 1.16 at cd = 0.00970, about 3.7° below stall; transition at x/c ≈ 0.05 (suction) and ≈ 0.97 (pressure) keeps the largely laminar pressure side near cd = 0.010 — five times cleaner than the aircraft CD,0 = 0.03, which also absorbs fuselage and interference drag.

The lift-slope chain of Eqs. (5)–(7) connects section to aircraft: 2π/rad rises to 7.72 with compressibility (XFoil), drops to 5.43 for the finite wing (wing-only AVL), and recovers to 5.92 for the full aircraft with the tail. Two caveats: the Karman-Tsien correction is approximate at Mach 0.5 (post-stall rows qualitative), and the section-level trim comparison is illustrative — the binding constraint is the pipeline's trim-angle-plus-incidence check. The two views still agree: stall margin 3.24° (pipeline) versus ≈ 3.7° (section).

Figure 1

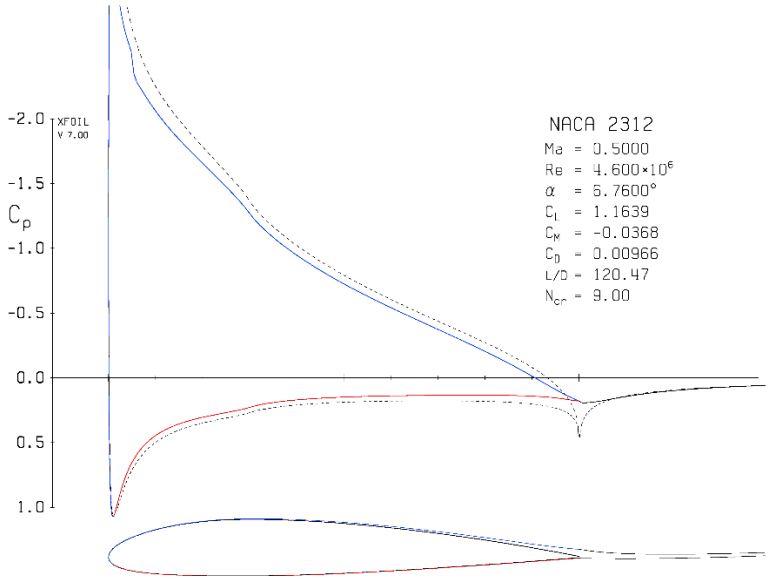
XFoil 7.00 Polar of the NACA 2312 at Re = 4.6×10⁶ and M = 0.5



Note. cl,max = 1.51 at α = 10.5

Figure 2

Section Pressure Distribution at the Cruise Trim Angle (α = 6.76°)



Note. Transition sits at x/c ≈ 0.05 on the suction side and ≈ 0.97 on the pressure side

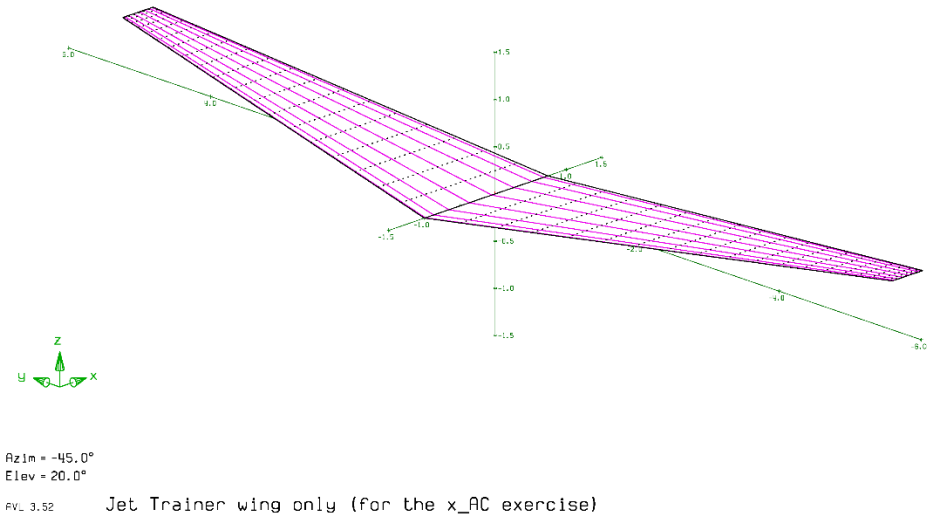
Wing Aerodynamic Center and Wing-Alone Stability

Following Eq. (8), the wing aerodynamic center was found from the wing-only model (Figure 3) two ways: a finite-difference sweep over α = 8.7–10.7° gives dCm/dCL = 0.0440 and xAC = −0.165 m, and the derivative ratio Cmα/CLα = 0.0395 gives −0.160 m — matching AVL's printed wing-only neutral point (−0.159914 m) to five decimals. The methods agree to about 0.5% of chord, consistent with finite-difference truncation over the 2° step.

The wing-only model used for the aerodynamic-center exercise of Eq. (8) is shown in Figure 3. It removes the horizontal and vertical tails while keeping every reference quantity (area, mean aerodynamic chord, and the moment reference point) unchanged, so that its pitching-moment slope isolates the wing contribution; the hardcopy pairs with the full-aircraft geometry of Figure 4 to show the simplification.

Figure 3

Wing-Only AVL Model for the Aerodynamic-Center Exercise



Note. Tails removed; reference quantities unchanged.

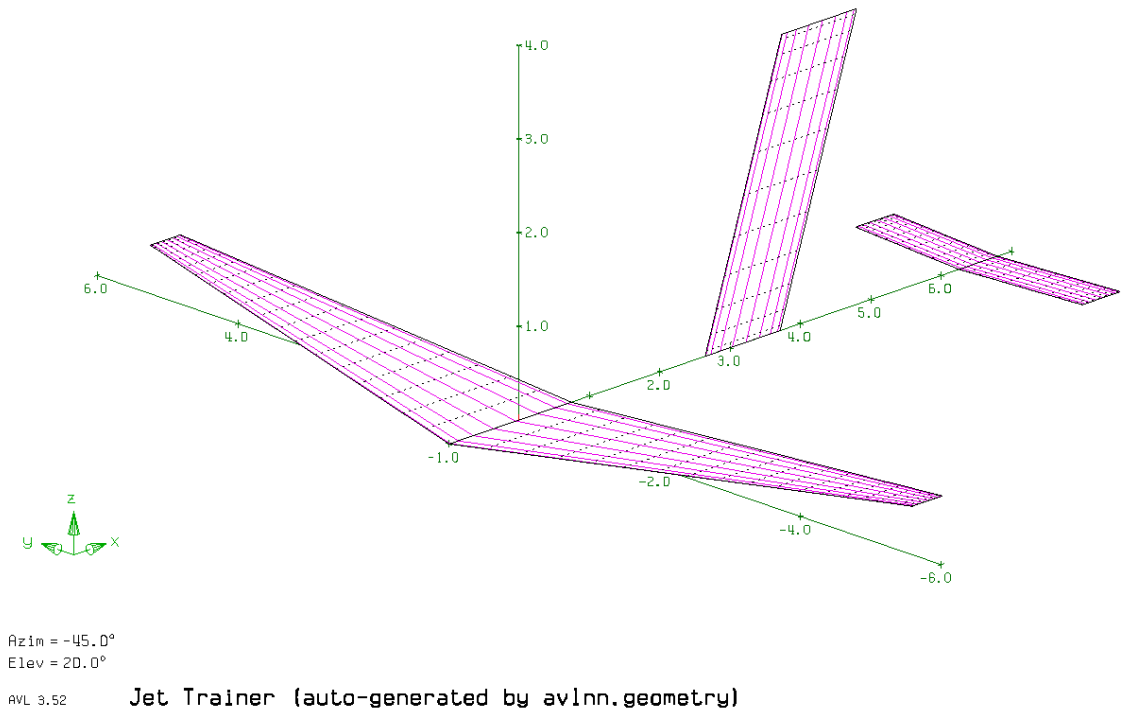
The aerodynamic center sits at 30.3% MAC, aft of the geometric quarter chord (25%, −0.224 m) — the expected sweep and finite-surface effect. Lying 0.048 m ahead of the center of gravity, the wing alone is statically unstable (dCm/dCL > 0); the tail moves the aircraft neutral point 0.49 m aft of it. At Mach 0, CLα falls from 5.43 to 4.87/rad (+11.5% at Mach 0.5) while xAC is essentially unchanged (−0.160 versus −0.160 m), as Prandtl-Glauert scales Cmα and CLα nearly equally.

Final Design, Trim, and Static Stability

All four seeds converged to the aircraft of Figure 4: wing aspect ratio 10.0 (upper bound), area 11.7 m², taper 0.247, quarter-chord sweep 8.83°, dihedral 4.52°, incidence −2.96°; span 10.8 m, mean aerodynamic chord 1.22 m, root and tip chords 1.74 m and 0.428 m. The horizontal tail sits at its maximum volume coefficient (0.800) and arm (6.50 m), the vertical tail at its maximum volume coefficient (0.080). The complete design vector is in Appendix A (Table 4).

Figure 4

AVL Geometry of the Final Design (Hardcopy Output)



Note. Vector hardcopy from AVL's geometry plot.

Table 2 lists the complete optimized design vector, including the bound status of each variable.

Table 2

Complete Optimized Design Vector (Eighteen Variables)

Design variable	Value	Bound status
Wing aspect ratio	10.0	upper bound
Wing taper ratio	0.247	—
Wing sweep (c/4)	8.83°	—
Wing dihedral	4.52°	—
Wing incidence	−2.96°	—
Wing area	11.7 m²	—
Wing root LE x	−0.995 m	—
Aileron span start	0.579 b/2	—
Aileron chord fraction	0.345	—
H-tail volume coefficient	0.800	upper bound
H-tail aspect ratio	5.92	—
H-tail sweep (c/4)	5.03°	—
H-tail arm	6.50 m	upper bound
V-tail volume coefficient	0.080	upper bound
V-tail aspect ratio	2.97	—
V-tail sweep (c/4)	34.5°	—
V-tail arm	3.02 m	—
Fuselage row spacing	1.50 m	lower bound

Note. Bound status marks variables that converged onto a limit of the optimizer's search space. Derived geometry: span 10.8 m; mean aerodynamic chord 1.22 m at y = 2.16 m (leading edge at x = −0.528 m); root chord 1.74 m; tip chord 0.428 m; center of gravity at x = −0.112 m; inertias Ixx = 5890, Iyy = 11,200, Izz = 16,400 kg·m².

Table 3 summarizes the trim state: α = 9.72° with elevator −8.07° gives CL = 0.844 and CD = 0.0527 — L/D = 14.1 — and the Trefftz-plane loading (Figure 5) gives span efficiency 0.992, near the elliptical ideal. The neutral point (+0.328 m) against the center of gravity (−0.112 m) yields a static margin of 0.362. The stall check, applied as trim angle plus wing incidence because the 10° limit holds anywhere on the wing, gives a worst section angle of 6.76° — a 3.24° margin.

Table 3

Trimmed Cruise State and Performance of the Final Design

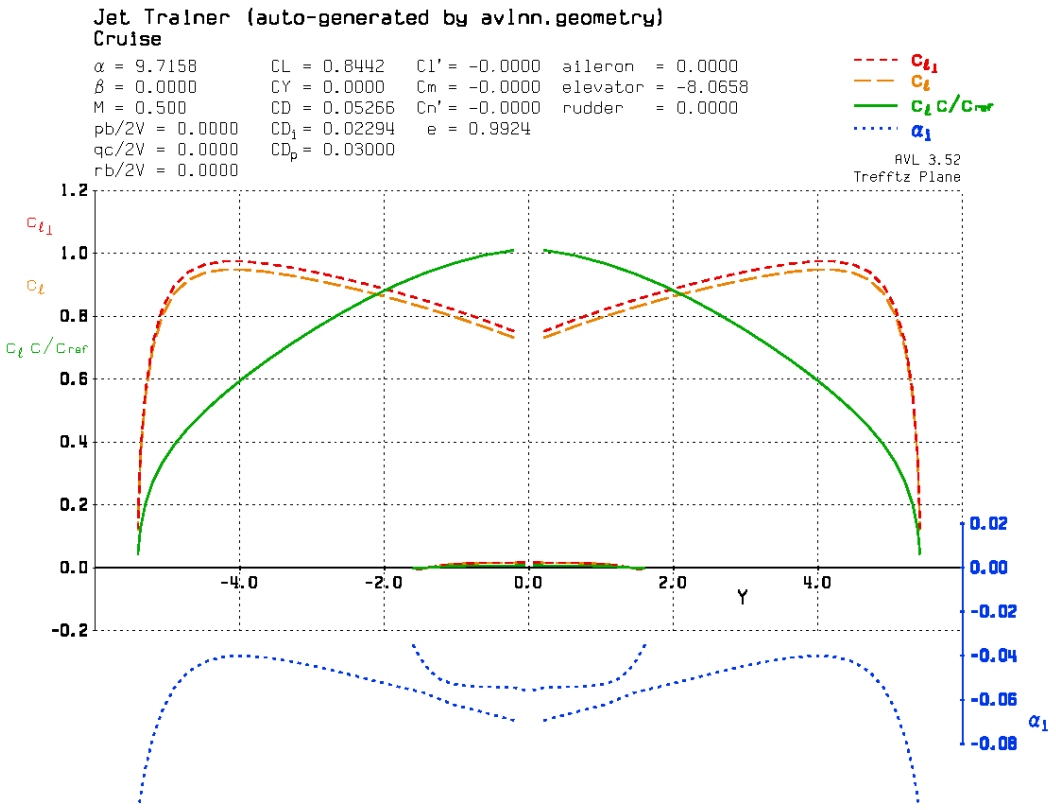
Parameter	Description	Value
CL	Trim lift coefficient	0.844
CD	Trim drag coefficient	0.0527
L/D	Cruise lift-to-drag ratio	14.1
e	Span efficiency (Trefftz plane)	0.992
α	Trim angle of attack	9.72°
δe	Elevator deflection at trim	−8.07°
xnp	Neutral point	+0.328 m
SM	Static margin	0.362

Stall margin	Trim α + wing incidence vs 10° limit	3.24° (worst section angle 6.76°)

Note. All values are re-verified AVL outputs at the cruise condition (M = 0.5, 11.0 km ISA). The complete design vector is given in Table 2.

Figure 5

Trefftz-Plane Loading at Cruise Trim



Note. Span efficiency e = 0.992.

Dynamic Stability

Table 4 lists the five classical modes from AVL's eigenvalue analysis, labels eigenvector-confirmed (Methodology). Four are stable: the short period ($\zeta = 0.196$ at 3.33 rad/s) and Dutch roll ($\zeta = 0.074$ at 3.29 rad/s) are adequately damped, the phugoid is slow and lightly damped ($\zeta = 0.101$), and the roll mode subsides with a 0.63 s time constant.

Table 4

Dynamic Modes of the Final Design (AVL Eigenvalue Analysis, Labels Eigenvector-Confirmed)

Mode	Eigenvalue (s^{-1})	ω_n (rad/s)	ζ or τ	Verdict
Short period	$-0.652 \pm 3.27i$	3.33	$\zeta = 0.196$	stable
Phugoid	$-0.00933 \pm 0.0919i$	0.0924	$\zeta = 0.101$	stable
Dutch roll	$-0.245 \pm 3.28i$	3.29	$\zeta = 0.074$	stable
Roll	-1.59	—	$\tau = 0.63$ s	stable
Spiral	$+0.00137$	—	$\tau = 729$ s	unstable, allowed ($\tau \geq 20$ s)

Note. Complex pairs are listed once; τ denotes the first-order time constant $1/|\lambda|$.

The spiral is the single exception: its root is marginally positive at $+0.00137\ s^{-1}$ (divergence time constant 729 s). The requirement allows a spiral divergence if its time constant

is at least 20 s, and at more than 36 times that threshold the design meets the dynamic-stability requirement in full.

Table 5 compares the approximations of Eqs. (9)–(15) with AVL. Short period is excellent (1.4% in frequency, 2% in damping), phugoid frequency within 1.8%, roll root within 6%, Dutch-roll frequency within 9% for a one-degree-of-freedom estimate. The analytical neutral point overshoots by ≈ 8% of chord (the classical formula neglects nonuniform downwash and tail position), and the phugoid damping is off by ≈ 2× — the one known-weak approximation, as Lanchester's model carries no drag-derivative detail. The spiral criterion agrees with AVL: $Cl\beta \cdot Cnr = 0.0406 < Clr \cdot Cn\beta = 0.0729$ (AVL prints the ratio 0.556).

Table 5

Classical Approximations Versus AVL Solutions

Quantity	Analytical	AVL	Difference
Wing xAC	25% MAC (−0.224 m)	30.3% MAC (−0.160 m)	sweep + finite AR
Neutral point	+0.420 m	+0.328 m	≈8% MAC
Static margin	0.44	0.362	analytical high
Short-period ωn	3.28 rad/s	3.33 rad/s	1.4%
Short-period ζ	0.200	0.196	2%
Phugoid ωn	0.0940 rad/s	0.0924 rad/s	1.8%
Phugoid ζ	0.044	0.101	≈2× (known-weak approximation)

Roll root	$-1.69\ s^{-1}$	$-1.59\ s^{-1}$	6%
Dutch-roll ωn	3.00 rad/s	3.29 rad/s	9% (1-DOF approximation)
Spiral	$Cl\beta \cdot Cnr < Clr \cdot Cn\beta \rightarrow$ unstable	$+0.00137\ s^{-1}$	agree

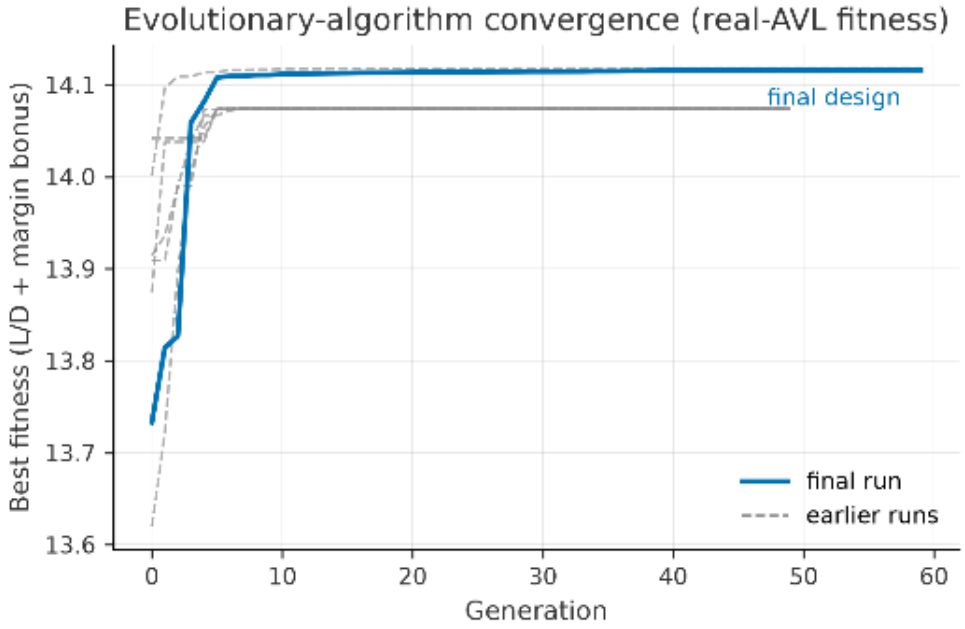
Note. Analytical values are evaluated at the cruise condition with the measured derivatives: aw = 5.433/rad from the wing-only model, at = 4.96/rad from Eq. (6), dε/dα = 0.346 from Eq. (7), and VH = 0.800.

Optimization Convergence and Surrogate Assessment

All four seeds reached a fitness of 14.074159 before the tie-break bonus — the analytic ceiling of Eq. (3), which at AR = 10 (e0 = 0.757) gives max L/D = 14.1 at CL* = 0.844 (S = 11.7 m^2). Agreement to six decimals from a stochastic search evidences a true optimum, not a merely feasible design (histories in Figure 6).

Figure 6

Evolutionary-Algorithm Best-Fitness Convergence Across Archived Runs



Note. All archived runs reach the analytic optimum. Run-time fitness histories predate the mass-file density fix; headline values in the text are re-verified post-fix outputs.

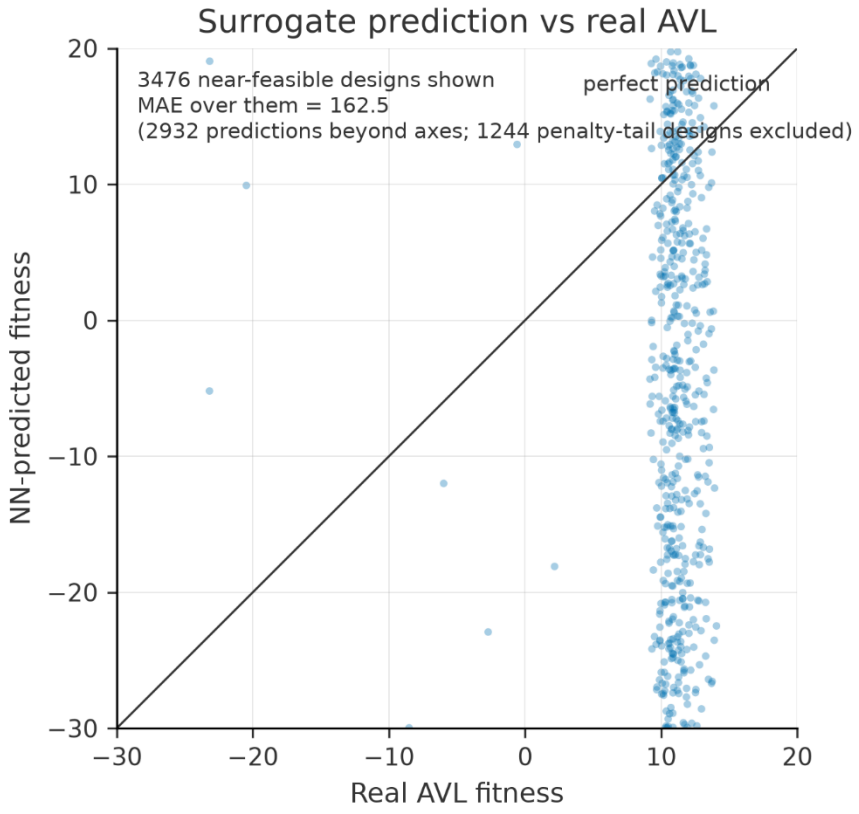
Bound-pinning is self-explaining: the ceiling rewards only aspect ratio and wing area, so the optimizer pushed AR and the tail volumes and horizontal-tail arm to their caps (where the tie-break is maximized) while the remaining variables stayed constraint-driven. AR = 10 is a modeling limit, not a recommendation — the objective has no structural penalty, and real trainers fly at AR ≈ 3.75–5.3.

The surrogate is a negative result with a clear diagnosis. Over 3476 near-feasible designs its mean absolute error was 162.5, with 2932 predictions beyond the ±30 parity band (Figure 7): the network cannot resolve the narrow feasible ridge the penalties carve, so the surrogate-assisted search converged off-optimum (L/D 13.0 versus 14.1 pure-AVL). The safeguard mattered —

every improvement was confirmed by real AVL before acceptance (Figure 8), so the surrogate could waste effort but not report a false optimum.

Figure 7

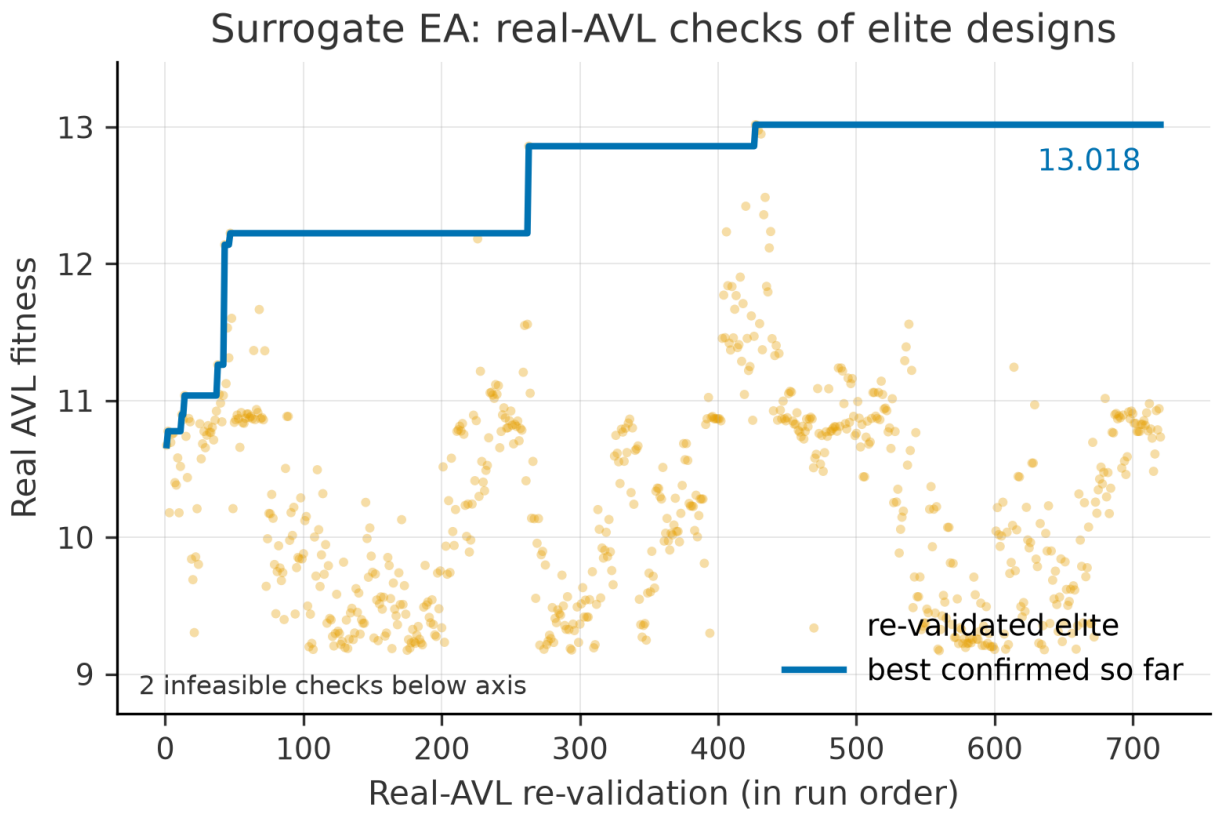
Surrogate-Predicted Versus Real-AVL Fitness (Parity Plot)



Note. Mean absolute error 162.5 over 3476 near-feasible designs; 2932 points fall beyond the ±30 band.

Figure 8

Real-AVL Re-Validation of Surrogate-Selected Elites During the Surrogate-Assisted Search



Conclusion

The project met every requirement: positive static margin (0.362), four of five dynamic modes stable with the spiral well inside its 20 s exception (τ = 729 s), positive stall margin (3.24°), and cruise L/D = 14.1 at the drag polar's analytic ceiling. The section study showed the 10° limit coinciding with the airfoil's own stall, and the wing-only exercise reproduced the classical story — a wing unstable alone, stabilized by its tail.

Automation was the real lesson: reproducibility — every number regenerates from the archived design vector — and confidence from four seeds agreeing to six decimals, a closed-form cross-check, two aerodynamic-center methods, and eigenvector-confirmed labels. It also taught caution: a plausible surrogate could not resolve the constraint ridge, and only real-solver re-validation kept it honest. Future work should add a structural model so the aspect-ratio bound no longer sets the optimum, and try surrogates that separate feasibility from performance.

References

Anderson, J. D. (2017). *Fundamentals of aerodynamics* (6th ed.). McGraw-Hill Education.

Drela, M. (1989). XFOIL: An analysis and design system for low Reynolds number airfoils. In T. J. Mueller (Ed.), *Low Reynolds number aerodynamics* (pp. 1–12). Springer. https://doi.org/10.1007/978-3-642-84010-4_1

Drela, M., & Youngren, H. (n.d.). AVL — Athena Vortex Lattice [Computer software]. Massachusetts Institute of Technology. http://web.mit.edu/drela/Public/web/avl/

Etkin, B., & Reid, L. D. (1996). *Dynamics of flight: Stability and control* (3rd ed.). John Wiley & Sons.

Nelson, R. C. (1998). *Flight stability and automatic control* (2nd ed.). McGraw-Hill.

Raymer, D. P. (2018). *Aircraft design: A conceptual approach* (6th ed.). American Institute of Aeronautics and Astronautics.

Appendix A – AVL and XFoil Submission Files

This appendix catalogues the submitted analysis files and explains what each contains, its format, and its source. The folder final_avl_files holds the final-design inputs and outputs: aircraft.avl (the AVL geometry file defining the wing, tails, and control surfaces), aircraft.mass (the point-mass distribution and inertias, with the mass-file density set to the cruise value), and aircraft.run (the trim run case); Instruction.txt reproduces the exact command sequences for the AVL session (stability derivatives, total forces, eigenvalue analysis) and for the two XFoil sessions (section polar and cruise pressure distribution); st_derivatives.txt, total_forces.txt, and eigenvalues.eig are the corresponding AVL text outputs from which every derivative, coefficient, and eigenvalue in this report was parsed; the geometry and Trefftz-plane hardcopies are provided in vector form (.ps/.pdf/.svg); wing_only.avl is the tails-removed model of the aerodynamic-center exercise, with reference quantities unchanged; and xfoil_polar.txt with the two XFoil plot files records the section study. The folder submission_outputs preserves the optimization evidence: per-generation logs (log.csv, surrogate_log.csv), seven archived evolutionary runs (ea_*.json), final_design.json (the frozen design vector), surrogate_ea_result.json, the 2000-sample dataset.csv, the trained network surrogate.pt, and the overnight run log.

Appendix B – Developed Code

The analysis and optimization pipeline was developed in Python; its modules generate the AVL input files, drive AVL and XFoil as subprocesses, parse their outputs, implement the evolutionary search and the surrogate training, and carry the 46-test verification suite described in the Methodology. These three modules are the algorithmic core of the pipeline described in the Methodology: the single funnel every design vector passes through (evaluate_design), the fitness function that turns AVL's output into the scalar the evolutionary algorithm optimizes (evaluate_objective), and the evolutionary algorithm itself (run_ea). The remaining pipeline files — geometry.py, massfile.py, runcase.py, and parse.py — are AVL file-format plumbing (text generation and regex parsing) and are described narratively in the Methodology rather than reproduced here.

B.1 — src/avlnn/evaluate.py

The pipeline's single entry point: every caller (the EA, dataset generation) passes a design vector through this one function, so the AVL round trip stays consistent everywhere and failures are always returned as an infeasible result rather than raised.

```
      """End-to-end: design vector -> AVL run -> objective fitness.

      Every caller (EA, dataset generation) should go through `evaluate_design` so
the
      geometry/mass/run-case/parse round trip stays consistent and AVL failures are
handled the same
      way everywhere (returned as an infeasible ObjectiveResult rather than raised).
      """
      from __future__ import annotations

      import tempfile
      from pathlib import Path

      from avlnn.avl_driver import AvlExecutionError, run_avl
      from avlnn.config import Constants
      from avlnn.geometry import build_avl_geometry
      from avlnn.massfile import build_avl_mass_file
      from avlnn.objective import ObjectiveResult, evaluate_objective,
infeasible_result
```

```
      from avlnn.parse import AvlParseError, parse_eigenvalues,
parse_stability_derivatives
      from avlnn.runcase import build_avl_run_case
      from avlnn.stability import evaluate_stability


      def evaluate_design(design: dict[str, float], c: Constants) ->
ObjectiveResult:
          """Runs the full AVL pipeline for one design vector in a scratch temp
directory."""
          geometry_text, geom = build_avl_geometry(design, c)
          mass_text = build_avl_mass_file(design, geom, c)
          run_text, _cl_target = build_avl_run_case(design, geom, c)

          with tempfile.TemporaryDirectory(prefix="avlnn_") as tmp:
              try:
                  result = run_avl(Path(tmp), geometry_text, mass_text, run_text)
              except AvlExecutionError as exc:
                  return infeasible_result(str(exc))

              if result.returncode != 0:
                  return infeasible_result(f"AVL exited with code
{result.returncode}: {result.stdout[-500:]}")

              if result.eig_text is None:
                  return infeasible_result(
                      "AVL did not write the eigenvalue (.eig) file -- MODE's
eigenmode "
                      "calculation likely refused to run; check the tail of AVL
stdout"
                  )

              try:
                  deriv = parse_stability_derivatives(result.stdout)
                  eigenvalues = parse_eigenvalues(result.eig_text)
                  stability = evaluate_stability(
                      x_np=deriv.x_np, x_cg=geom.x_cg_b_m, cref=geom.cref_m,
                      alpha_trim_deg=deriv.alpha_deg,
                      wing_incidence_deg=design["wing_incidence_deg"],
                      eigenvalues=eigenvalues, c=c,
                  )
              except (AvlParseError, ValueError) as exc:
                  return infeasible_result(str(exc))

              return evaluate_objective(deriv, stability, design, geom.sref_m2, c)
```

B.2 — src/avlnn/objective.py

Combines AVL-derived stability and performance data into the single scalar fitness the EA optimizes: cruise L/D minus proportional penalties for every violated constraint, plus a small saturating stability-margin bonus that breaks ties among equally efficient feasible designs (see the module docstring for why the bonus is necessary).

```
      """Combines AVL-derived stability/performance data into a single scalar
fitness for the EA.

      Fitness = cruise L/D, minus penalties proportional to how far each hard
constraint (static
```

```
      margin, each dynamic mode, stall margin, thrust-vs-drag feasibility) is
violated. Proportional
      (not flat) penalties give the EA a gradient to climb even from infeasible
starting genomes.

      Fully feasible designs additionally get a small saturating stability-margin
bonus (capped at
      TIEBREAK_WEIGHT, i.e. well under 1% of a typical L/D). Rationale: in this
problem max L/D
      depends only on aspect ratio and wing area, so the many remaining design
variables (sweep,
      dihedral, tails, ...) affect fitness solely through the constraints -- without
a tie-break the
      EA has no reason to prefer a robustly stable design over one that scrapes past
every check.
      The tanh saturation keeps huge margins (e.g. a spiral time constant of
hundreds of seconds)
      from buying more than the cap, so the bonus orders equal-L/D designs without
ever trading
      meaningfully against L/D itself.
      """
      from __future__ import annotations

      import dataclasses
      import math

      from avlnn.atmosphere import isa
      from avlnn.config import Constants
      from avlnn.derived import available_thrust_N, drag_coefficient
      from avlnn.parse import StabilityDerivatives
      from avlnn.stability import StabilityReport

      INFEASIBLE_FITNESS = -1000.0
      PENALTY_WEIGHT = 100.0
      TIEBREAK_WEIGHT = 0.05

      # Characteristic "comfortable margin" scales: tanh(margin/scale) is ~0.76 at
the scale value
      # and saturates toward 1 beyond ~2x it. Units match each check's margin
(static margin in
      # fractions of cref, stall in degrees, modes in 1/s damping -- spiral in
seconds of tau slack).
      _MARGIN_SCALES = {
          "static": 0.10,
          "stall_deg": 3.0,
          "phugoid": 0.05,
          "short_period": 2.0,
          "dutch_roll": 0.5,
          "roll": 3.0,
          "spiral": 60.0,
      }


      def _stability_margin_bonus(stability: StabilityReport) -> float:
          terms = [
              math.tanh(stability.static_margin / _MARGIN_SCALES["static"]),
              math.tanh(stability.stall_margin_deg / _MARGIN_SCALES["stall_deg"]),
          ]
          for name, check in stability.mode_checks.items():
              terms.append(math.tanh(check.margin / _MARGIN_SCALES[name]))
          return TIEBREAK_WEIGHT * sum(terms) / len(terms)


      @dataclasses.dataclass(frozen=True)
      class ObjectiveResult:
          fitness: float
```

```
            l_over_d: float
            thrust_margin_N: float
            stability: StabilityReport | None
            infeasible_reason: str | None = None


        def evaluate_objective(
            deriv: StabilityDerivatives,
            stability: StabilityReport,
            design: dict[str, float],
            wing_area_m2: float,
            c: Constants,
        ) -> ObjectiveResult:
            atm = isa(c.mission.altitude_cruise_m, c.atmosphere)
            velocity = c.mission.mach_cruise * atm.speed_of_sound_m_s
            q = 0.5 * atm.density_kg_m3 * velocity**2
            weight_N = c.mass.total_kg * c.atmosphere.g0_m_s2

            cd = drag_coefficient(deriv.cl, design["wing_aspect_ratio"], c.aero.cd0)
            l_over_d = deriv.cl / cd if cd > 0 else 0.0

            drag_N = cd * q * wing_area_m2
            thrust_avail_N = available_thrust_N(
                c.propulsion.static_thrust_to_weight_sl, weight_N,
c.mission.mach_cruise,
                c.propulsion.thrust_zero_mach, atm.density_kg_m3,
c.atmosphere.rho0_sea_level_kg_m3,
            )
            thrust_margin_N = thrust_avail_N - drag_N

            penalty = 0.0
            if not stability.static_margin_passed:
                penalty += PENALTY_WEIGHT * abs(stability.static_margin)
            if not stability.stall_margin_passed:
                penalty += PENALTY_WEIGHT * abs(stability.stall_margin_deg)
            for mode in stability.mode_checks.values():
                if not mode.passed:
                    penalty += PENALTY_WEIGHT * abs(mode.margin)
            if thrust_margin_N < 0:
                penalty += PENALTY_WEIGHT * abs(thrust_margin_N) / weight_N

            # Bonus only for fully feasible designs; infeasible ones follow the pure
penalty
            # gradient toward feasibility first.
            bonus = _stability_margin_bonus(stability) if penalty == 0.0 else 0.0

            return ObjectiveResult(
                fitness=l_over_d + bonus - penalty, l_over_d=l_over_d,
                thrust_margin_N=thrust_margin_N, stability=stability,
            )


        def infeasible_result(reason: str) -> ObjectiveResult:
            """Used when AVL fails to run/converge/parse -- keeps the EA/dataset gen
from crashing."""
            return ObjectiveResult(
                fitness=INFEASIBLE_FITNESS, l_over_d=0.0, thrust_margin_N=float("-
inf"),
                stability=None, infeasible_reason=reason,
            )
```

B.3 — src/avlnn/ea.py

The evolutionary algorithm itself: tournament selection, BLX-α crossover, Gaussian mutation, elitism. Deliberately decoupled from AVL — it only requires an evaluate_fn(x) -> float callable — which is what let the same engine drive both the pure-AVL search and the NN-surrogate-assisted search without duplicating any code.

```
      """A real-valued evolutionary algorithm: tournament selection, BLX-alpha
crossover, Gaussian
      mutation, elitism.

      Deliberately decoupled from AVL -- it only needs an `evaluate_fn(x:
np.ndarray) -> float`
      fitness callable, so this exact engine drives both the pure-AVL search
(scripts/run_ea.py) and
      the NN-surrogate search (surrogate/surrogate_ea.py) without duplication.
      """
      from __future__ import annotations

      import concurrent.futures
      import dataclasses
      from typing import Callable

      import numpy as np

      from avlnn.design_space import DesignSpace

      EvaluateFn = Callable[[np.ndarray], float]


      @dataclasses.dataclass
      class EaConfig:
          population_size: int = 40
          n_generations: int = 50
          elite_fraction: float = 0.1
          tournament_size: int = 3
          crossover_prob: float = 0.8
          blx_alpha: float = 0.3
          mutation_prob: float = 0.15
          mutation_sigma_frac: float = 0.1   # mutation stdev as a fraction of each
variable's range
          n_workers: int = 1
          seed: int | None = None


      @dataclasses.dataclass
      class GenerationRecord:
          generation: int
          best_fitness: float
          mean_fitness: float
          best_design: np.ndarray


      @dataclasses.dataclass
      class EaResult:
          best_design: np.ndarray
          best_fitness: float
          history: list[GenerationRecord]
          final_population: np.ndarray
          final_fitness: np.ndarray

          def top_k(self, k: int) -> np.ndarray:
```

```
            """The k distinct-fittest individuals from the final population (for
re-validation)."""
            order = np.argsort(self.final_fitness)[::-1][:k]
            return self.final_population[order]


    def _evaluate_population(pop: np.ndarray, evaluate_fn: EvaluateFn, n_workers:
int) -> np.ndarray:
        if n_workers <= 1:
            return np.array([evaluate_fn(ind) for ind in pop])
        with concurrent.futures.ProcessPoolExecutor(max_workers=n_workers) as
pool:
            return np.array(list(pool.map(evaluate_fn, pop)))


    def _tournament_select(
        pop: np.ndarray, fitness: np.ndarray, k: int, rng: np.random.Generator,
    ) -> np.ndarray:
        idx = rng.integers(0, len(pop), size=k)
        winner = idx[np.argmax(fitness[idx])]
        return pop[winner]


    def _blx_alpha_crossover(
        p1: np.ndarray, p2: np.ndarray, alpha: float, rng: np.random.Generator,
    ) -> np.ndarray:
        lo, hi = np.minimum(p1, p2), np.maximum(p1, p2)
        span = hi - lo
        return rng.uniform(lo - alpha * span, hi + alpha * span)


    def _mutate(
        ind: np.ndarray, space: DesignSpace, prob: float, sigma_frac: float, rng:
np.random.Generator,
    ) -> np.ndarray:
        mask = rng.random(len(ind)) < prob
        sigma = sigma_frac * (space.upper - space.lower)
        noise = rng.normal(0.0, sigma)
        return space.clip(np.where(mask, ind + noise, ind))


    def run_ea(
        space: DesignSpace,
        evaluate_fn: EvaluateFn,
        config: EaConfig,
        on_generation: Callable[[GenerationRecord], None] | None = None,
        initial_population: np.ndarray | None = None,
    ) -> EaResult:
        """`initial_population` (shape (population_size, n_dims)) seeds the
population instead of
        uniform random sampling -- used by the surrogate-assisted EA to carry the
evolved
        population across surrogate-retraining blocks instead of restarting each
block."""
        rng = np.random.default_rng(config.seed)
        if initial_population is not None:
            pop = space.clip(np.asarray(initial_population, dtype=float))
            expected_shape = (config.population_size, space.n_dims)
            if pop.shape != expected_shape:
                raise ValueError(f"initial_population shape {pop.shape} !=
{expected_shape}")
        else:
            pop = space.sample_uniform(config.population_size, rng)
        fitness = _evaluate_population(pop, evaluate_fn, config.n_workers)

        n_elite = max(1, round(config.elite_fraction * config.population_size))
```

```
            history: list[GenerationRecord] = []

            for gen in range(config.n_generations):
                elite_idx = np.argsort(fitness)[::-1][:n_elite]
                elites = pop[elite_idx]
                elite_fitness = fitness[elite_idx]

                children = []
                while len(children) < config.population_size - n_elite:
                    p1 = _tournament_select(pop, fitness, config.tournament_size, rng)
                    p2 = _tournament_select(pop, fitness, config.tournament_size, rng)
                    child = (
                        _blx_alpha_crossover(p1, p2, config.blx_alpha, rng)
                        if rng.random() < config.crossover_prob
                        else p1.copy()
                    )
                    children.append(_mutate(child, space, config.mutation_prob,
config.mutation_sigma_frac, rng))

                # Elites' genomes are unchanged, and evaluate_fn (AVL or the
surrogate) is
                # deterministic, so re-running them would just waste evaluations --
only the new
                # children need fresh fitness values.
                children_arr = np.array(children)
                children_fitness = _evaluate_population(children_arr, evaluate_fn,
config.n_workers)
                pop = np.vstack([elites, children_arr])
                fitness = np.concatenate([elite_fitness, children_fitness])

                best_i = int(np.argmax(fitness))
                record = GenerationRecord(
                    generation=gen, best_fitness=float(fitness[best_i]),
                    mean_fitness=float(np.mean(fitness)),
best_design=pop[best_i].copy(),
                )
                history.append(record)
                if on_generation is not None:
                    on_generation(record)

            best_i = int(np.argmax(fitness))
            return EaResult(
                best_design=pop[best_i], best_fitness=float(fitness[best_i]),
history=history,
                final_population=pop, final_fitness=fitness,
            )
```